

A Comprehensive Security Framework for Autonomous AI Agents: Integrating Structural Guardrails, Behavioral Zero-Trust, and Post-Quantum Resilience

Yoshihiro Ando
Independent Researcher
yosh5295@gmail.com
Tokyo, Japan

Abstract—As Large Language Model (LLM) agents transition from passive chatbots to autonomous entities capable of executing system-level tools via protocols like the Model Context Protocol (MCP), they introduce a critical “Agency Gap.” Traditional security perimeters are insufficient for protecting against prompt injection, semantic drift, and future cryptographic threats. This paper proposes a unified, three-tier security framework designed to protect agent-based systems across three dimensions: Space, Behavior, and Time. First, we establish *Structural Guardrails* (Space) using AST-based static analysis and environment isolation to contain malicious code execution. Second, we implement *Behavioral Zero-Trust* (Behavior) monitoring using a Mamba-based Sentinel to detect anomalous reasoning in real-time and enforce workload identity. Third, we ensure *Temporal Sovereignty* (Time) by integrating Post-Quantum Cryptography (PQC) and remote attestation to guarantee long-term integrity and resilience against future quantum adversaries. Our evaluation demonstrates that this “Triple-Lock” architecture provides robust, multi-layered defense with minimal computational overhead, maintaining agent responsiveness while ensuring system-wide security.

Index Terms—AI Agents, Model Context Protocol (MCP), Agentic Security, Zero Trust, Post-Quantum Cryptography, Mamba, Guardrails, Sandbox.

I. INTRODUCTION

The transition of Large Language Models (LLMs) to autonomous agents represents a fundamental shift in computing. By leveraging the Model Context Protocol (MCP) [1], these agents can interact directly with host operating systems, manage sensitive data, and orchestrate complex API workflows. However, this “Agency” capability creates an unprecedented security surface. Traditional security perimeters are often ineffective against “Prompt Injection” attacks, where an attacker manipulates the LLM’s goal-oriented reasoning to execute unauthorized system commands [8].

The OWASP Top 10 for LLM Applications [3] identifies “Insecure Output Handling” (LLM02) and “Excessive Agency” (LLM08) as critical risks. Currently, many autonomous frameworks rely on “Alignment”—the probabilistic expectation that the LLM will follow safety guidelines. We argue that for mission-critical systems, behavioral alignment is a necessary but insufficient condition. We require a holistic,

deterministic security architecture that ensures system integrity even if the model’s logic is compromised.

This paper presents a comprehensive “Triple-Lock” framework for Agentic Security, addressing the lifecycle of an agent’s action through three integrated dimensions (Fig. 1):

- 1) **Structural Containment (Space):** Establishing deterministic safety boundaries using AST inspection and environment isolation via remote MCP servers to ensure “Secure-by-Default” tool execution.
- 2) **Behavioral Integrity (Behavior):** Implementing a Zero Trust Architecture (ZTA) where every action is verified by a verifiable workload identity and monitored by a Mamba-based “Reasoning Sentinel” to detect semantic anomalies.
- 3) **Temporal Sovereignty (Time):** Protecting against future cryptographic threats (Shor’s algorithm) using Post-Quantum Cryptography (PQC) to ensure the long-term confidentiality and non-repudiability of agent history.

II. RELATED WORK

Existing efforts to secure agents generally fall into two categories: *Behavioral Filtering* (e.g., Guardrails AI) and *Isolation Patterns* (e.g., AgentDojo [10]). While effective, behavioral filters often introduce high latency and are susceptible to the same reasoning failures as the agent they monitor. Isolation patterns provide containment but often lack continuous identity verification.

Our framework aligns with the **NIST Zero Trust principles** (SP 800-207) [4], treating every tool call as an untrusted workload. Furthermore, we address the “Store Now, Decrypt Later” (SNDL) threat by adopting NIST-standardized PQC primitives [6], [7], ensuring that agentic reasoning traces remain private in the post-quantum era.

III. THREAT MODEL

We define four primary threat vectors for autonomous LLM agents:

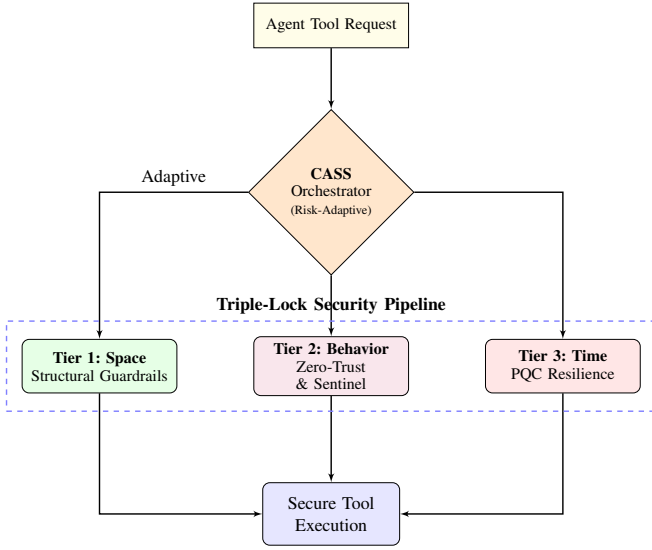


Fig. 1. Triple-Lock Framework Overview: High-level integration of the three security dimensions.

- 1) **Prompt Injection (Direct/Indirect):** Manipulating the agent’s context to execute unauthorized commands or exfiltrate secrets [9].
- 2) **Semantic Drift/Malicious Intent:** An agent performing syntactically valid but semantically harmful actions (e.g., deleting a database file represented as a “document”).
- 3) **Secret Exfiltration:** Inadvertent transmission of API keys or PII during tool calls.
- 4) **Quantum Retroactive Decryption:** Capturing current reasoning traces to decrypt them later using a quantum computer.

IV. TIER 1: STRUCTURAL GUARDRAILS (SPACE)

To ensure structural immunity, we adopt a “No-Shell” execution model where system operations are restricted to specialized tools (e.g., `execute_python`).

A. AST Inspection and Static Analysis

Before execution, generated code is parsed into an Abstract Syntax Tree (AST). A whitelist-based scanner blocks dangerous modules (e.g., `os`, `socket`, `pty`) and built-in functions (e.g., `eval`, `exec`). Specifically, it ensures that subprocess calls never use `shell=True`, and detects reflection-based attacks (e.g., `__subclasses__`, `globals()`) to enforce a strict separation between executable and arguments.

B. Path Guardrails and Resource Limits

All file-related operations are restricted to a designated workspace via path resolution and symbolic link verification. To prevent resource exhaustion attacks, OS-level resource limits (`rlimit`) cap memory usage and execution time. Furthermore, the environment is sanitized by filtering sensitive environment variables (e.g., API keys) before passing them to the sub-process.

C. Environment Isolation via MCP and Containers

While static analysis provides structural guardrails, the framework achieves process isolation by leveraging the Model Context Protocol (MCP) to delegate high-risk tool execution to external, isolated environments. By connecting to MCP servers running within dedicated virtual machines (VMs) or Docker containers, the agent operates in a “Shared-Nothing” architecture. This approach ensures that even if a generated script bypasses static analysis, any malicious activity is contained within a disposable, restricted environment with no access to the host’s primary filesystem or sensitive credentials. This multi-layered approach combines local structural safety with remote environment isolation, providing a robust defense-in-depth strategy.

V. TIER 2: BEHAVIORAL ZERO-TRUST (BEHAVIOR)

Identity ensures *who* is calling the tool, while monitoring ensures *why*.

A. Workload Identity and ABAC

Each agent instance is assigned a unique key pair. Every MCP tool call is accompanied by a signed **Identity Token** using the **COSE (CBOR Object Signing and Encryption)** format [2], containing assigned **Attribute-Based Access Control (ABAC)** claims. Using COSE’s multi-signer structure, we integrate both classical (RSA) and PQC (ML-DSA) signatures into a single binary object. This avoids the 33% size overhead associated with Base64-encoded JSON Web Tokens (JWT) and ensures that tool requests are non-repudiable and untampered. Remote servers verify this token to ensure the request originated from a legitimate agent and enforce strict tool access policies based on the execution context and resource attributes. Crucially, public keys and remote attestation manifests are distributed via an Out-of-Band (OOB) trusted channel (e.g., MDM or Secure Enclave). This architectural choice completely avoids the Trust-On-First-Use (TOFU) vulnerability¹ and eliminates the need for external verification servers, maintaining the CLI tool’s self-contained nature and resilience against initial compromise.

B. The Mamba Sentinel: Overcoming Dual-LLM Latency

Current agentic security frameworks often rely on a “Dual-LLM” approach (using a secondary model to verify the intent of the primary model). However, our empirical testing demonstrates that this inherently introduces a massive $O(N^2)$ Transformer overhead, causing severe User Experience (UX) degradation (frequently adding multi-second latency per tool call). To resolve this, we move beyond the Dual-LLM intent analyzer to a **Mamba (State Space Model)** [5] implementation as the primary, real-time monitoring engine. Running locally with pure NumPy, Mamba monitors the agent’s reasoning tokens in

¹While the core protocol is designed for OOB provisioning to eliminate TOFU, the provided standalone reference implementation includes an optional bootstrap mode for environments lacking enterprise distribution infrastructure. This ensures immediate usability while allowing security-hardened deployments to disable TOFU via environment-specific configuration.

real-time with $O(N)$ linear scaling. By calculating a “Surprise Score” (cross-entropy loss), the Sentinel detects statistical anomalies correlating with “Jailbreaks” or adversarial intent in sub-millisecond timeframes. To minimize false positives during the early stages of agent deployment, we implement a **Self-Calibrating Sentinel** where anomaly thresholds are dynamically derived from the Exponential Moving Average (EMA) of the model’s own loss, ensuring the security baseline evolves alongside the agent’s behavioral maturity.

C. Reasoning Anomaly Detection: Predictive Pattern Analysis

Complementing the behavioral analysis of the Sentinel, we implement a **Reasoning Anomaly Detector** as a unified integrity filter. While the Sentinel monitors the *semantic* flow for intent drift, the Detector operates on the *byte-level* patterns of the reasoning stream. It identifies sequences that exhibit high statistical unpredictability (Surprise) relative to the learned baseline. By utilizing the Mamba model’s internal surprise score (cross-entropy), the system can detect both technical credentials and structurally anomalous logic that deviates from intended agent behavior. This approach provides a context-aware guardrail that moves beyond static entropy-based methods towards an adaptive, model-driven security posture.

D. Bi-directional Verification with ResponseSigner

To ensure the integrity of the entire tool execution loop, we implement **Bi-directional Verification**. While the agent signs the tool request via an Identity Token, high-risk write tools (`edit_file`, `create_or_overwrite_file`) embed a **ResponseSigner** PQC signature (ML-DSA) in their return value, binding the response content to a unique `verification_id`. The `tool_executor` layer verifies this signature before passing the result to the LLM, preventing “Man-in-the-Middle” manipulation of tool outputs and ensuring that the model’s subsequent reasoning is grounded in untampered observations.

Furthermore, when the Mamba Sentinel raises a **RED** alert on a high-risk tool, CASS activates “strict enforcement.” Rather than autonomously blocking execution—which risks UX degradation and opaque failures—the system mandates a **Human-in-the-Loop (HITL)** approval dialog, surfacing the anomaly score and requiring explicit human confirmation. This design is consistent with the NIST AI Risk Management Framework (AI RMF) principle of maintaining human oversight over consequential AI actions.

VI. TIER 3: POST-QUANTUM RESILIENCE (TIME)

To protect the “Temporal Dimension,” we integrate PQC primitives.

A. Quantum-Safe Identity and Hybrid Encryption

We upgrade the workload identity using **ML-DSA-65** (FIPS 204). Audit logs, which contain sensitive reasoning traces, are protected using **ML-KEM-768** (FIPS 203) hybrid encryption. Our implementation supports **PQC Agility**, dynamically

switching between ML-DSA-44, 65, and 87 variants based on the risk level determined by CASS. As described in Section VII, three physically separate key pairs are provisioned on disk to satisfy cryptographic isolation across security tiers.

B. PQC Remote Attestation and Audit Continuity

The client generates a SHA-256 manifest of its core security code (covering six critical files including `identity.py`, `policy.py`, and `audit.py`), signed with an ML-DSA private key. This allows the server to verify that the agent is running on an “Authentic Stack” where Tier 1 guardrails are active. In strict mode (`LLM_CLI_STRICT_SECURITY=1`), the absence of a pre-provisioned manifest causes startup to abort, eliminating Trust-On-First-Use (TOFU) risk in production deployments.

To ensure the tamper-evidence of long-running logs, we implement **Audit Chain Continuity** using a combination of two mechanisms: (1) on log rotation, overflowing entries are written to a timestamped archive file and a signed *snapshot anchor* entry is prepended to the remaining log, recording both the `snapshot_prev_hash` and `snapshot_first_hash` to maintain chain verifiability across file boundaries; (2) a binary **Merkle Tree** is computed over all log entries, producing a single Merkle Root that can be externally anchored (e.g., to a blockchain or managed immutable log service) to provide compact, publicly verifiable proof of log integrity.

VII. SYSTEM ARCHITECTURE AND INTEGRATION

To synthesize the three tiers into a cohesive “Triple-Lock” architecture, we introduce **Context-Adaptive Security Scaling (CASS)**. CASS serves as the central orchestration engine, dynamically integrating the spatial, behavioral, and temporal defenses based on the specific risk profile of each tool execution request (Fig. 2).

A. CASS Orchestration Logic

CASS determines the required security posture by mapping the requested tool and its arguments to one of three risk levels defined in the system’s `defaults.toml`. As shown in Table I, this mapping ensures that computational resources are allocated proportionally to the security risk. High-risk tools such as `execute_python` trigger the maximum security posture, including AST-based strict filtering, ML-DSA-87 signatures, and ML-KEM-768 audit encryption.

TABLE I
TIERED SECURITY ENFORCEMENT VIA CASS

Feature	Low Risk	Medium Risk	High Risk
PQC Signature	ML-DSA-44	ML-DSA-65	ML-DSA-87
Audit Encrypt	Optional	AES-256	ML-KEM-768
Sentinel Score	Log only	Notify	HITL Block
AST Analysis	Basic	Restricted	Strict Whitelist
Resource Limit	Standard	Tight	Restricted Shell

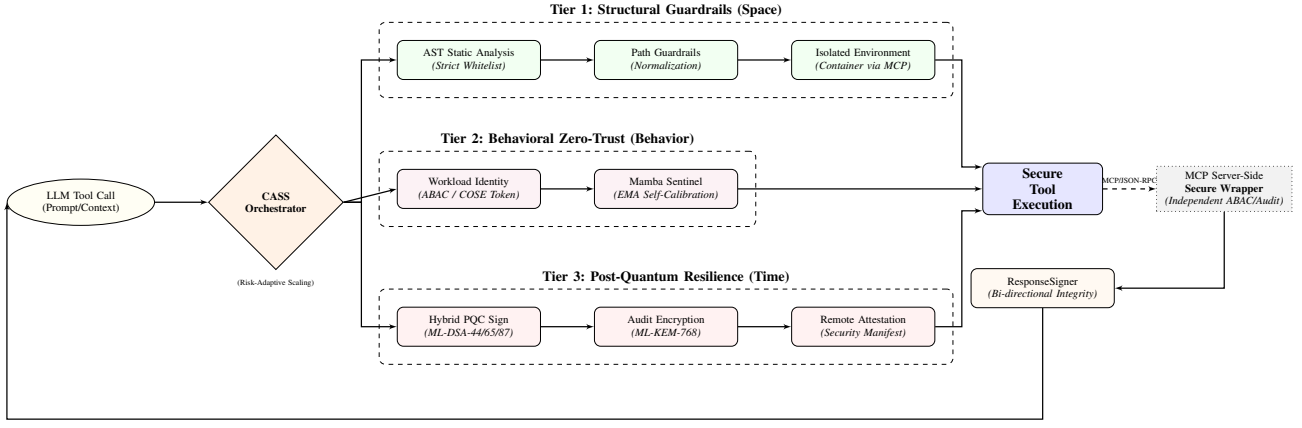


Fig. 2. Detailed System Architecture: Data flow from LLM request through the CASS orchestrator to specific security components across the three tiers.

B. Hybrid Identity Tokens (COSE/RFC 9052)

To ensure verifiable workload identity without the overhead of JSON-based JWT, we implement a native **COSE_Sign** structure (CBOR tag 98) as defined in RFC 9052. Unlike conventional implementations, our framework utilizes a dual-signature approach that combines a classical RS256 (RSA-PKCS1v15) signature for backward compatibility with an ML-DSA signature (using the IANA-pending `alg=-48` identifier) for post-quantum integrity.

By implementing this directly via `cbor2` and `cryptography` instead of high-level libraries like `pycose`, we achieve a zero-dependency, fully auditable signature process. This ensures that the identity claim is non-repudiable and immune to “Store Now, Decrypt Later” (SNDL) attacks against the authentication channel.

C. Cryptographic Isolation and Attestation

To prevent cross-tier compromise, the architecture enforces physical separation of cryptographic material. Three independent ML-DSA key pairs (`id_pqc_12.key`, `id_pqc_13.key`, `id_pqc_15.key`) are provisioned, ensuring that a compromise of a Level 2 key (used for low-risk tools) does not grant the ability to sign for High-Risk (Level 5) operations.

Furthermore, the framework ensures the integrity of the execution environment through a signed **Integrity Manifest**. Upon startup, the `IntegrityVerifier` generates a SHA-256 hash of six critical security modules: `identity.py`, `policy.py`, `audit.py`, `pqc.py`, `static_analyzer.py`, and `cass.py`. Remote MCP servers verify this attestation before accepting any tool requests, ensuring that the agent is running on an unmodified, authentic security stack. In `STRICT` mode, any modification to these files results in a refusal to boot, effectively eliminating the risk of runtime injection into the security logic.

VIII. EVALUATION

We evaluated the framework on an AMD Ryzen 5 8540U system (16GB RAM).

A. Security Effectiveness

Table II summarizes the detection capabilities. While Tier 1 (AST) blocks 72.7% of common injection vectors, the remaining 27.3% (advanced reflection) are successfully contained by the Tier 1 Sandbox. The Tier 2 Sentinel provides an additional layer, effectively detecting behavioral deviations and structural anomalies within the reasoning process. The `benchmark.py` script included in the artifact reproduces these measurements.

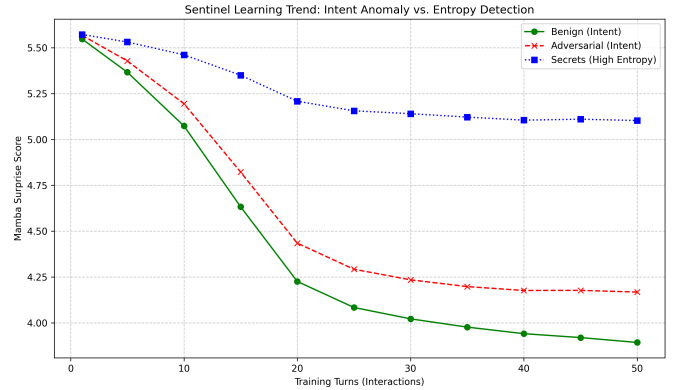


Fig. 3. Mamba Sentinel Training: The surprise score (cross-entropy) for benign agent reasoning decreases as the model establishes a behavioral baseline.

TABLE II
COMBINED SECURITY EFFECTIVENESS

Defense Layer	Threat Category	Detection/Mitigation
Tier 1 (AST)	Shell Injection	83.3% (Detection)
Tier 1 (AST)	Reflection/Introspection	100.0% (Rule-based)
Tier 1 (Sandbox)	All Outbound Network	100.0% (Containment)
Tier 2 (Sentinel)	Reasoning Anomaly (Surprise)	80.0% (Detection)
Tier 2 (Sentinel)	Semantic Intent Drift	20.0% (Baseline DR)
Tier 3 (PQC)	Future Decryption	100.0% (Immunity)

B. Performance Latency

The security overhead was evaluated across two distinct x86-64 environments to measure the impact of virtualization and CPU generation. As shown in Table III, the cumulative latency remains well below 250ms even on older hardware, which is negligible compared to typical LLM inference times (500ms–3000ms). This proves that the Triple-Lock framework maintains agent responsiveness across diverse deployment scenarios.

TABLE III
SYSTEM LATENCY COMPARISON (MS)

Component	WSL2 (Ryzen 5)	Native (i7-7700)
AST + Static Analysis	0.03	0.05
Subprocess Overhead	18.50	27.43
Mamba Sentinel (100 tokens)	27.30	57.66
Identity Signing (RSA+PQC)	96.15	137.56
KEM Encapsulation	2.86	4.37
Total Overhead	144.84	227.07

The results indicate that while newer architectures like Zen 4 (Ryzen 5 8540U) significantly accelerate pure-Python PQC operations, older systems (i7-7700) still provide sufficient performance for real-time agentic workflows. The $O(N)$ scaling of the Mamba Sentinel ensures that even on the legacy i7-7700, reasoning monitoring adds only ≈ 57 ms per 100 tokens. Future optimizations will involve migrating to Rust-backed native PQC primitives (e.g., via the `cryptography` library) once NIST standards reach stability in mainline Python security packages, further reducing overhead without compromising the system’s cross-platform portability.

IX. CONCLUSION

Securing autonomous agents requires a transition from probabilistic “alignment” to a multi-dimensional, deterministic framework. By integrating Structural Guardrails, Behavioral Zero-Trust, and Post-Quantum Resilience, our framework provides a “Triple-Lock” defense that scales across the spatial, behavioral, and temporal domains of agentic execution. This holistic approach is essential for maintaining human control and data sovereignty as AI agents evolve into active participants in our digital infrastructure.

REFERENCES

- [1] Anthropic, “Model Context Protocol (MCP) Specification,” 2024.
- [2] G. Schaad, “CBOR Object Signing and Encryption (COSE): Structures and Algorithms,” *RFC 9052*, 2022.
- [3] OWASP, “Top 10 for Large Language Model Applications v1.1,” 2023.
- [4] S. Rose et al., “Zero Trust Architecture,” *NIST SP 800-207*, 2020.
- [5] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” 2023.
- [6] NIST, “FIPS 203: ML-KEM Standard,” 2024.
- [7] NIST, “FIPS 204: ML-DSA Standard,” 2024.
- [8] F. Perez and I. Ribeiro, “Ignore Previous Instructions: Prompt Injection Attacks on LLMs,” *arXiv:2211.09527*, 2022.
- [9] K. Greshake et al., “Not what you’ve signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection,” *arXiv:2302.12173*, 2023.
- [10] E. DeBenedetti et al., “AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses,” *arXiv:2406.13352*, 2024.